

COMP3308 Assignment 2 Part 2: Report

Student 1: <SID>

Student 2: <SID>

1 Introduction

The aim of this study was to investigate and compare the performance of different machine learning algorithms on a heart failure dataset. We used a 10-fold stratified cross validation method and compared the performance of our own built algorithms and Weka models, including baseline methods, KNN, decision trees and ensembles of trees. We also improved our KNN and DT models by making slight changes to the models.

2 Data

The dataset used in this study was the heart failure dataset, which contains 273 patient instances. Each instance represents a patient record with several medical attributes related to heart failure. The dataset contains 11 predictive attributes and 1 class attribute. The predictive attributes include factors such as age, anaemia status, CPK, diabetes status, ejection fraction, blood pressure, platelet count, serum creatinine level, serum sodium level, sex, and smoking status.

Most attributes were categorical. The class attribute contains two possible classes: died and survived. The dataset is moderately imbalanced because the number of survived instances is larger than the number of died instances. This imbalance makes the classification task more challenging, which is more difficult to predict the died class.

To evaluate classifier performance, 10-fold stratified cross-validation was used. The dataset was divided into 10 approximately equal folds while preserving the original class distribution within each fold. During evaluation, one fold was used as the testing set and the remaining nine folds were combined as the training set. This process was repeated 10

times so that every instance was used exactly once for testing. The final performance measures were calculated by averaging the results across all folds.

3 The KNN+ and DT+ Algorithms

3.1 KNN+

In the original MyKNN implementation, each neighbour had the same voting power regardless of how close it was to the testing instance. This means that a very close neighbour and a more distant neighbour could affect the final prediction equally.

MyKNN+ modified the standard KNN algorithm by using distance-weighted voting. Instead of counting each neighbour as one vote, each neighbour was given a weight based on its distance from the testing instance. The weight was calculated as:

$$\text{weight} = \frac{1}{\text{distance} + 0.25}$$

In this case, the 0-distance neighbour will have 4 votes, which can significantly affect the result of prediction, while the 3-distance has only approximate 0.3 votes.

The motivation for this modification was that closer instances are more likely to be similar to the testing instance and therefore should have stronger influence on the classification result. Two patients with same status are more likely to have the same result.

3.2 DT+

The standard MyDT algorithm used Information Gain to select the best attribute and didn't do any pruning, which produced a very large tree and learned very specific cases, leading to overfitting.

MyDT+ modified the standard decision tree algorithm by using Gain Ratio to replace Information Gain as the criterion of splitting and applying pre-pruning through maximum depth limitation. After looking for the best maximum depth, we found that the main important attributes are actually in the top 2 levels, so we limited the maximum depth to 2, which makes the tree simple and not overfitting.

The motivation for this modification was to improve generalisation. The original MyDT created many small branches that fitted the training data too closely, while MyDT+ produced a much simpler tree. This was important as the dataset is relatively small, that deep branches could easily be based on few examples.

In the final version, MyDT+ used a shallow tree structure, where only the most important attributes were used for prediction. This made the model easier to interpret and more suitable for a small medical dataset.

4 Results and Discussion

4.1 Results

Predictive performance – accuracy and other suitable measures

Measure	ZeroR	1R	KNN	NB	MLP	SVM	MyKNN	MyKNN+
Accuracy (%)	67.033	71.7949	70.696	71.0623	63.7363	72.5275	63.3700	67.3993
Weighted Precision	-	0.710	0.690	0.695	0.630	0.718	0.628	0.658
Weighted Recall	0.670	0.718	0.707	0.711	0.637	0.725	0.634	0.674
Weighted F1	-	0.713	0.690	0.697	0.633	0.720	0.630	0.663

Measure	DT unpruned	DT pruned	MyDT	MyDT+	Bagg	Boost	RF
Accuracy (%)	65.9341	72.1612	63.7363	71.0623	69.5971	67.3993	68.1319
Weighted Precision	0.660	0.707	0.651	0.693	0.674	0.670	0.661
Weighted Recall	0.659	0.722	0.637	0.711	0.696	0.674	0.681
Weighted F1	0.660	0.697	0.643	0.685	0.669	0.672	0.664

The values for the Weka classifiers are taken from `Run information.docx`. The values for MyKNN, MyKNN+, MyDT and MyDT+ were regenerated from the submitted Python classifiers.

4.1.1 Accuracy

For accuracy aspect, SMO achieved the highest accuracy at approximately 72.53%, followed by OneR with 71.8%. Our own models My DT+ also achieved 71.0%, beating Bagging (69.6%) and Boosting (69.4%) with a simple structure. Random forest got 68.1%, while ZeroR got 67.0%. My KNN was the lowest with accuracy of 63%, and My KNN+

improved to 67.4%. This result indicated that the sample was too small to generate complex algorithms, as ensembles of trees were getting 67%-69%, which is lower than the simple algorithms.

4.1.2 Precision

For precision, SMO had the strongest overall weighted precision at 71.8%, OneR also performed well with 71%. MyDT+ achieved a weighted precision of 69.3%, which was higher than MyDT's 65.1%, showing that pruning made the decision tree less noisy and more reliable. Naive Bayes achieved 69.5%, similar to MyDT+, while KNN achieved 69%. However, Random Forest and Bagging did not perform strongly in precision, with weighted precision around 66.1% and 67.4%, which showed that complex models didn't fit the dataset.

For the died class specifically, precision is important because it shows how many patients predicted as died were actually died. Weka DT had a relatively high died precision of 63.5%, while MyDT+ had 60.8% and SMO had 59.5%. This means these models were more careful when predicting the minority died class, giving fewer false died predictions.

4.1.3 Recall

For recall, SMO achieved relatively strong recall at 52.2% for died, meaning they detected more actual died cases than many other classifiers. OneR also performed well with died recall of 51.1%. Other models are weak on predicting died recall. This suggests that some models achieved good overall accuracy, however, they were weak at identifying high risk died cases.

Specifically, our MyDT+ showed a trade-off. It improved overall accuracy and precision compared with MyDT, but its died recall dropped to 34.4%. This means pruning made the model simpler and more accurate overall, but it also removed some deeper branches that helped detect died cases.

4.2 Discussion

4.2.1 Comparison of MyKNN, MyKNN+, MyDT and MyDT+

MyDT+ was developed to improve the performance of the original MyDT algorithm by reducing overfitting and improving attribute selection. We made two main modifications. In MyDT+, we used Gain Ratio rather than Information Gain as the standard of selecting branches. The other change is we applied pruning to MyDT+.

The motivation for using Gain Ratio instead of Information Gain was to reduce the bias towards highly branching attributes, which helps the algorithm select attributes that provide more meaningful and balanced splits. Pruning was introduced to prevent the tree from becoming too deep and too specialised to the training data.

The results displayed a significant improve in performance of the model. When the maximum depth of tree was limited to 3, the accuracy of the prediction achieved 69.1%, which is apparently higher than the MyDT of 63.8%. Moreover, when we doing further pruning by limiting the maximum depth, it showed a better prediction accuracy. The result was 71.0% when the maximum depth was set to 2 and 72.1% at 1.

This indicates that limiting tree complexity successfully improved model performance. The strongest predictive information was already split in the first 2 levels, especially in the top level, while additional splits only introduced noise and led to overfitting.

These results suggest that the model is not becoming better by increasing the complexity of trees. Simpler decision trees generalised better than highly detailed trees. Therefore, the modifications introduced in DT+ were effective in reducing overfitting, although the optimal performance depended strongly on selecting an appropriate pruning depth.

4.2.2 Comparison with Weka Classifiers

Although both MyDT and Weka DT were based on the same fundamental decision tree learning principles, their performance differed. My DT has an accuracy at 63.8% while Weka J48 unpruned has 65.9%. This is mainly because in My DT, we used a simple recursive splitting process based on Information Gain while Weka used other criteria to improve performance.

To compare My DT+ with Weka J48 pruned algorithm, both models achieved accuracy

of 72,1%. My DT+ only used basic pre-pruning by limiting the maximum tree depth to 1 or 2, while Weka J48 used more advanced approaches. However, as the sample space is really small with only 273 instances, the models showed similar performance.

4.2.3 Effect of Pruning in Weka Decision Trees

The unpruned tree gives a larger structure with 124 nodes and 79 leaves, making the tree highly specific. Some of the branches even has only one instances, which makes the tree more like memorising the data rather than predicting. As a result, the model learned some noise and is overfitted, leading to a low performance of the prediction.

In comparison, the pruned tree has only 11 leaves and 17 nodes. It is a clear structure that reducing the probability of overfitting. Performance of the model is significantly better than the previous unpruned tree, with an accuracy at 72.16%.

These results demonstrate that the pruning approach reduces learning noise in the dataset and finally improves the performance of predicting model.

4.2.4 Comparison of Weka Tree-Based Classifiers

The comparison between Weka's tree-based classifiers showed that the pruned J48 decision tree achieved the best overall performance on the heart-failure dataset. The pruned J48 model achieved approximately 72.16% accuracy, outperforming Bagging, Random Forest and Boosting.

Although the ensembles of trees methods usually give a better performance than single trees, in this case, the single pruned tree got the highest accuracy. Bagging achieved approximately 71.43% accuracy, Random Forest achieved around 68.13%, and AdaBoost achieved approximately 67.40%.

One possible reason is that the dataset was relatively small, containing only 273 instances, which makes the result somehow random. In addition, as discussed above, the class in this dataset is mainly determined by the top-level split and all other attributes don't change the result a lot. Overall, the results suggest that a relatively simple pruned decision tree was more suitable for this small medical dataset than more complex ensemble-based tree classifiers.

5 Conclusion

This project compared several machine learning algorithms on the heart-failure dataset using 10-fold cross-validation. The results demonstrated that model complexity does not always lead to better performance. Ensemble methods such as Random Forest, Bagging, and Boosting achieved only moderate results, suggesting that the dataset may have been too small for complex tree models.

For KNN, we found that giving closer neighbours higher weight can improve classification quality by improving our KNN+ model. For DT, our improvement on the model showed that pre-pruning was useful to give a better prediction, as it reduced the effects of overfitting.

For future work, several improvements could be explored. On the one hand, we may use larger dataset to train our model, which may help it perform better and reduce overfitting. On the other hand, we can apply more advanced techniques on our models, like multilayer perceptron and post-pruning.

6 Reflection

Student 1: <SID>

Write your reflection here.

Student 2: <SID>

Write your reflection here.

7 Acknowledgement of AI Use

Student 1: <SID>

Write your AI-use acknowledgement here, or state: No AI tools were used in this assignment.

Student 2: <SID>

Write your AI-use acknowledgement here, or state: No AI tools were used in this assignment.

Appendix A: Code for KNN+

```
import csv

# MyKNN+ uses the same Hamming-style distance as MyKNN, but it gives
# closer neighbours more influence during voting.

def classify_knn(training_filename, testing_filename, k):
    """Classify each testing row using distance-weighted k-nearest neighbours."""

    training_data = []

    testing_data = []

    result = []

    with open(training_filename, "r") as f:

        reader = csv.reader(f)

        for row in reader:

            training_data.append([x.strip() for x in row])

    with open(testing_filename, "r") as f:

        reader = csv.reader(f)

        for row in reader:

            testing_data.append([x.strip() for x in row])

    for row in testing_data:

        distances = []

        # Count how many categorical attributes differ between two patients.
        for index, data in enumerate(training_data):

            distance = 0

            for i in range(len(row)):

                if row[i] != data[i]:

                    distance += 1

            distances.append([distance, index, data[-1]])
```



```

# Sort by distance first. The original row index is a deterministic
# tie-breaker when two training instances have the same distance.
distances.sort(key=lambda x: (x[0], x[1]))

nearest = distances[:k]

died_score = 0

survived_score = 0

for item in nearest:

    distance = item[0]

    label = item[-1]

    # DT/KNN+ modification: closer neighbours get larger weights.
    # The +0.25 avoids division by zero and gives exact matches
    # a strong but finite vote.
    weight = 1 / (distance + 0.25)

    if label == "died":

        died_score += weight

    else:

        survived_score += weight

if survived_score > died_score:

    result.append("survived")

else:

    result.append("died")

return result

def read_folds(folds_filename):
    """Read the provided stratified folds file into a list of folds."""

    folds = []

    current_fold = []

    with open(folds_filename, "r") as f:

        reader = csv.reader(f)

        for row in reader:

            if len(row) == 0:

                continue

            if row[0].startswith("fold"):

                if current_fold != []:


```

```

        folds.append(current_fold)

        current_fold = []

    else:

        current_fold.append(row)

    if current_fold != []:

        folds.append(current_fold)

    return folds

def write_csv(filename, rows):
    """Write temporary train/test files used by the existing classifier API."""

    with open(filename, "w", newline="") as f:

        writer = csv.writer(f)

        writer.writerows(rows)

def evaluate_knn_10fold(folds_filename, k):
    """Evaluate MyKNN+ using stratified 10-fold cross-validation."""

    folds = read_folds(folds_filename)

    accuracies = []

    for i in range(10):

        testing_with_class = folds[i]

        training = []

        for j in range(10):

            if j != i:

                training.extend(folds[j])

        # The testing file must not contain the class label.
        testing_without_class = []

        for row in testing_with_class:

            testing_without_class.append(row[:-1])

        write_csv("train_temp.csv", training)

        write_csv("test_temp.csv", testing_without_class)

        predictions = classify_knn("train_temp.csv", "test_temp.csv", k)

        correct = 0

        for prediction, actual_row in zip(predictions, testing_with_class):

```

```

        actual_class = actual_row[-1]

        if prediction == actual_class:

            correct += 1

        accuracy = correct / len(testing_with_class)

        accuracies.append(accuracy)

        #print("Fold", i + 1, "accuracy:", accuracy)

    average_accuracy = sum(accuracies) / len(accuracies)

    print("Average accuracy:", average_accuracy)

    return average_accuracy

evaluate_knn_10fold("heart-folds.csv", 1)
evaluate_knn_10fold("heart-folds.csv", 2)
evaluate_knn_10fold("heart-folds.csv", 3)
evaluate_knn_10fold("heart-folds.csv", 4)
evaluate_knn_10fold("heart-folds.csv", 5)
evaluate_knn_10fold("heart-folds.csv", 6)

```

Appendix B: Code for DT+

```

import csv
import math

# MyDT+ is the modified decision tree. It uses gain ratio for splitting and
# pre-pruning with a maximum depth to reduce overfitting on the small dataset.

ATTRIBUTE_NAMES = [
    "age",
    "anaemia",
    "CPK",
    "diabetes",
    "ejection_fraction",
    "high_blood_pressure",
    "platelets",
    "serum_creatinine",
    "serum_sodium",
    "sex",
    "smoking"
]

CLASS_LABELS = ["died", "survived"]

def read_csv(filename):
    """Read a CSV file and trim whitespace from every cell."""
    data = []
    with open(filename, "r") as f:

```

```

        reader = csv.reader(f)
        for row in reader:
            data.append([x.strip() for x in row])
    return data

def read_folds(folds_filename):
    """Read the provided stratified folds from heart-folds.csv."""
    folds = []
    current_fold = []

    with open(folds_filename, "r") as f:
        reader = csv.reader(f)

        for row in reader:
            if len(row) == 0:
                continue

            if row[0].startswith("fold"):
                if current_fold != []:
                    folds.append(current_fold)
                    current_fold = []
            else:
                current_fold.append([x.strip() for x in row])

        if current_fold != []:
            folds.append(current_fold)

    return folds

def class_counts(rows):
    """Count how many training rows belong to each class."""
    counts = {}

    for label in CLASS_LABELS:
        counts[label] = 0

    for row in rows:
        counts[row[-1]] += 1

    return counts

def majority_class(rows, default_class="died"):
    """Return the majority class, using default_class for empty subsets."""
    if len(rows) == 0:
        return default_class

    counts = class_counts(rows)

    if counts["died"] >= counts["survived"]:
        return "died"
    else:
        return "survived"

def all_same_class(rows):
    """Check whether all rows in a subset have the same class label."""

```

```

first_class = rows[0][-1]

for row in rows:
    if row[-1] != first_class:
        return False

return True

def entropy(rows):
    """Calculate class entropy for a subset of rows."""
    counts = class_counts(rows)
    total = len(rows)
    result = 0

    for label in CLASS_LABELS:
        if counts[label] > 0:
            p = counts[label] / total
            result -= p * math.log2(p)

    return result

def group_by_attribute(rows, attribute_index):
    """Group rows by one attribute value before calculating a split score."""
    groups = {}

    for row in rows:
        value = row[attribute_index]

        if value not in groups:
            groups[value] = []

        groups[value].append(row)

    return groups

def information_gain(rows, attribute_index):
    """Calculate information gain for one candidate splitting attribute."""
    parent_entropy = entropy(rows)
    groups = group_by_attribute(rows, attribute_index)
    weighted_entropy = 0
    total = len(rows)

    for group in groups.values():
        weighted_entropy += len(group) / total * entropy(group)

    return parent_entropy - weighted_entropy

def split_information(rows, attribute_index):
    """Calculate split information, the normalising term in gain ratio."""
    groups = group_by_attribute(rows, attribute_index)
    total = len(rows)
    split_info = 0

    for group in groups.values():
        p = len(group) / total

```

```

        if p > 0:
            split_info -= p * math.log2(p)

    return split_info

def gain_ratio(rows, attribute_index):
    """Calculate gain ratio to reduce bias toward many-valued attributes."""
    split_info = split_information(rows, attribute_index)

    if split_info == 0:
        return 0

    return information_gain(rows, attribute_index) / split_info

def best_attribute(rows, attributes):
    """Choose the attribute with the highest gain ratio."""
    best_attr = attributes[0]
    best_score = gain_ratio(rows, best_attr)

    for attr in attributes[1:]:
        score = gain_ratio(rows, attr)

        if score > best_score:
            best_score = score
            best_attr = attr

    return best_attr

def make_leaf(rows, default_class):
    """Create a leaf node and store Weka-style count/error information."""
    prediction = majority_class(rows, default_class)
    counts = class_counts(rows)
    total = len(rows)
    errors = total - counts[prediction]

    return {
        "leaf": True,
        "prediction": prediction,
        "count": total,
        "errors": errors,
        "counts": counts
    }

def build_tree(rows, attributes, default_class, depth, max_depth):
    """Recursively build the pre-pruned MyDT+ decision tree."""
    if len(rows) == 0:
        return make_leaf(rows, default_class)

    if all_same_class(rows):
        return make_leaf(rows, default_class)

    if len(attributes) == 0:
        return make_leaf(rows, default_class)

```

```

# Pre-pruning: stop before the tree becomes too specific to training data.
if depth >= max_depth:
    return make_leaf(rows, default_class)

current_majority = majority_class(rows, default_class)
attr = best_attribute(rows, attributes)

tree = {
    "leaf": False,
    "attribute": attr,
    "majority": current_majority,
    "count": len(rows),
    "children": {}
}

values = []

for row in rows:
    if row[attr] not in values:
        values.append(row[attr])

remaining_attributes = []

for a in attributes:
    if a != attr:
        remaining_attributes.append(a)

for value in values:
    subset = []

    for row in rows:
        if row[attr] == value:
            subset.append(row)

    tree["children"][value] = build_tree(
        subset,
        remaining_attributes,
        current_majority,
        depth + 1,
        max_depth
    )

return tree

def predict(tree, row):
    """Predict one row by following matching branches in the tree."""
    if tree["leaf"]:
        return tree["prediction"]

    attr = tree["attribute"]
    value = row[attr]

    if value not in tree["children"]:
        return tree["majority"]

    return predict(tree["children"][value], row)

```

```

def train_dt_plus(rows):
    """Train MyDT+ on the supplied rows."""
    attributes = []

    for i in range(len(rows[0]) - 1):
        attributes.append(i)

    return build_tree(
        rows,
        attributes,
        majority_class(rows),
        0,
        2
    )

def classify_dt_plus(training_filename, testing_filename=None, return_tree=False):
    """Train MyDT+ and return predictions, or the tree if requested."""
    training_data = read_csv(training_filename)
    tree = train_dt_plus(training_data)

    if return_tree:
        return tree

    testing_data = read_csv(testing_filename)
    predictions = []

    for row in testing_data:
        predictions.append(predict(tree, row))

    return predictions

def format_count(number):
    return str(float(number))

def leaf_text(leaf):
    text = leaf["prediction"] + " (" + format_count(leaf["count"])

    if leaf["errors"] > 0:
        text += "/" + format_count(leaf["errors"])

    text += ")"
    return text

def print_tree_diagram(tree, indent=""):
    """Return a text-based tree diagram similar to Weka's J48 output."""
    if tree["leaf"]:
        return indent + leaf_text(tree) + "\n"

    attr_index = tree["attribute"]
    attr_name = ATTRIBUTE_NAMES[attr_index]
    result = ""

    for value in tree["children"]:
        child = tree["children"][value]

```



```

        if child["leaf"]:
            result += indent + attr_name + " = " + value + ": "
            result += leaf_text(child) + "\n"
        else:
            result += indent + attr_name + " = " + value + "\n"
            result += print_tree_diagram(child, indent + "|  ")

    return result

def count_leaves(tree):
    if tree["leaf"]:
        return 1

    total = 0

    for child in tree["children"].values():
        total += count_leaves(child)

    return total

def tree_size(tree):
    if tree["leaf"]:
        return 1

    total = 1

    for child in tree["children"].values():
        total += tree_size(child)

    return total

def evaluate_predictions(predictions, actual_labels):
    """Calculate accuracy, precision, recall, F1 and the confusion matrix."""
    total = len(actual_labels)
    correct = 0
    matrix = {}

    for actual in CLASS_LABELS:
        matrix[actual] = {}
        for predicted in CLASS_LABELS:
            matrix[actual][predicted] = 0

    for prediction, actual in zip(predictions, actual_labels):
        matrix[actual][prediction] += 1

        if prediction == actual:
            correct += 1

    class_metrics = {}

    for label in CLASS_LABELS:
        tp = matrix[label][label]
        fp = 0
        fn = 0

        for other in CLASS_LABELS:

```

```

        if other != label:
            fp += matrix[other][label]
            fn += matrix[label][other]

    if tp + fp == 0:
        precision = 0
    else:
        precision = tp / (tp + fp)

    if tp + fn == 0:
        recall = 0
    else:
        recall = tp / (tp + fn)

    if precision + recall == 0:
        f1 = 0
    else:
        f1 = 2 * precision * recall / (precision + recall)

    class_metrics[label] = {
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "support": tp + fn
    }

weighted_precision = 0
weighted_recall = 0
weighted_f1 = 0

for label in CLASS_LABELS:
    weight = class_metrics[label]["support"] / total
    weighted_precision += class_metrics[label]["precision"] * weight
    weighted_recall += class_metrics[label]["recall"] * weight
    weighted_f1 += class_metrics[label]["f1"] * weight

return {
    "accuracy": correct / total,
    "correct": correct,
    "incorrect": total - correct,
    "total": total,
    "matrix": matrix,
    "class_metrics": class_metrics,
    "weighted_precision": weighted_precision,
    "weighted_recall": weighted_recall,
    "weighted_f1": weighted_f1
}

def evaluate_dt_plus_10fold(folds_filename):
    """Evaluate MyDT+ using stratified 10-fold cross-validation."""
    folds = read_folds(folds_filename)
    accuracies = []
    all_predictions = []
    all_actual_labels = []

    for i in range(10):
        testing_with_class = folds[i]
        training = []

```

```

    for j in range(10):
        if j != i:
            training.extend(folds[j])

    tree = train_dt_plus(training)
    predictions = []

    for row in testing_with_class:
        predictions.append(predict(tree, row[:-1]))

    actual_labels = []

    for row in testing_with_class:
        actual_labels.append(row[-1])

    metrics = evaluate_predictions(predictions, actual_labels)
    accuracies.append(metrics["accuracy"])
    all_predictions.extend(predictions)
    all_actual_labels.extend(actual_labels)

    print("Fold", i + 1, "accuracy:", round(metrics["accuracy"], 4))

overall_metrics = evaluate_predictions(all_predictions, all_actual_labels)
overall_metrics["average_fold_accuracy"] = sum(accuracies) / len(accuracies)

return overall_metrics

def format_metric(number):
    return f"{number:.3f}"

def print_matrix_row(matrix, actual_label, letter):
    died_count = matrix[actual_label]["died"]
    survived_count = matrix[actual_label]["survived"]
    print(
        f"{died_count:4d}{survived_count:4d} | "
        + letter
        + " = "
        + actual_label
    )

def print_evaluation_report(metrics):
    print("=== MyDT+ 10-fold cross-validation ===")
    print("Correctly Classified Instances", metrics["correct"],
          " ", round(metrics["accuracy"] * 100, 4), "%")
    print("Incorrectly Classified Instances", metrics["incorrect"],
          " ", round((1 - metrics["accuracy"]) * 100, 4), "%")
    print("Average fold accuracy:",
          round(metrics["average_fold_accuracy"], 4))
    print("Total Number of Instances", metrics["total"])
    print()

    print("=== Detailed Accuracy By Class ===")
    print("Precision Recall F-Measure Class")

    for label in CLASS_LABELS:

```

```

    row = metrics["class_metrics"][label]
    print(
        "
        "
        + format_metric(row["precision"])
        + "
        "
        + format_metric(row["recall"])
        + "
        "
        + format_metric(row["f1"])
        + "
        "
        + label
    )

print(
    "Weighted Avg.
    "
    + format_metric(metrics["weighted_precision"])
    + "
    "
    + format_metric(metrics["weighted_recall"])
    + "
    "
    + format_metric(metrics["weighted_f1"])
)
print()

print("=== Confusion Matrix ===")
print("  a  b  <-- classified as")
print_matrix_row(metrics["matrix"], "died", "a")
print_matrix_row(metrics["matrix"], "survived", "b")
print()

def all_training_rows_from_folds(folds_filename):
    folds = read_folds(folds_filename)
    rows = []

    for fold in folds:
        rows.extend(fold)

    return rows

def print_full_training_tree(tree):
    print("=== Classifier model (full training set) ===")
    print("MyDT+ tree")
    print("-----")
    print()
    print(print_tree_diagram(tree), end="")
    print()
    print("Number of Leaves : \t", count_leaves(tree))
    print("Size of the tree : \t", tree_size(tree))
    print()

def write_full_training_tree(filename, tree):
    with open(filename, "w") as f:
        f.write("=== Classifier model (full training set) ===\n")
        f.write("MyDT+ tree\n")
        f.write("-----\n\n")
        f.write(print_tree_diagram(tree))
        f.write("\n")
        f.write("Number of Leaves : \t" + str(count_leaves(tree)) + "\n")

```

```

        f.write("Size of the tree : \t" + str(tree_size(tree)) + "\n")

def run_report(folds_filename):
    print("=== Run information ===")
    print("Scheme:      MyDT+")
    print("Relation:     heart-failure")
    print("Test mode:     10-fold cross-validation")
    print()

    metrics = evaluate_dt_plus_10fold(folds_filename)
    print()
    print_evaluation_report(metrics)

    training_rows = all_training_rows_from_folds(folds_filename)
    full_tree = train_dt_plus(training_rows)
    print("=== Appendix: Decision Tree Diagram ===")
    print()
    print_full_training_tree(full_tree)
    write_full_training_tree("mydtplus_tree.txt", full_tree)
    print("Tree diagram saved to mydtplus_tree.txt")

    return metrics

if __name__ == "__main__":
    run_report("heart-folds.csv")

```

Appendix C: MyDT Decision Tree Diagram

=== Classifier model (full training set) ===

MyDT tree

```

serum_creatinine = high
|  serum_sodium = normal
|  |  high_blood_pressure = no
|  |  |  diabetes = yes
|  |  |  |  age = [61-70]: died (2.0)
|  |  |  |  age = [51-60]
|  |  |  |  |  sex = F: died (2.0)
|  |  |  |  |  sex = M: survived (4.0)
|  |  |  |  age = 70plus: died (1.0)
|  |  |  |  age = [40-50]
|  |  |  |  |  smoking = no: survived (1.0)
|  |  |  |  |  smoking = yes: died (1.0)
|  |  |  |  diabetes = no
|  |  |  |  |  smoking = no
|  |  |  |  |  age = [51-60]
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  sex = M: died (1.0)
|  |  |  |  |  |  |  sex = F: survived (1.0)
|  |  |  |  |  |  anaemia = no: survived (2.0)
|  |  |  |  |  age = [40-50]: died (1.0)
|  |  |  |  |  age = 70plus

```

```

| | | | | | CPK = normal: survived (1.0)
| | | | | | CPK = high: died (1.0)
| | | | | | age = [61-70]
| | | | | | ejection_fraction = normal
| | | | | | | anaemia = no: died (1.0)
| | | | | | | anaemia = yes: survived (1.0)
| | | | | | ejection_fraction = low: survived (2.0)
| | | | | | ejection_fraction = mildly-low: survived (1.0)
| | | | | smoking = yes: survived (6.0)
| | high_blood_pressure = yes
| | | age = 70plus: died (3.0)
| | | age = [51-60]
| | | | anaemia = yes
| | | | sex = F: died (2.0)
| | | | sex = M: survived (1.0)
| | | | anaemia = no: survived (2.0)
| | | age = [61-70]: died (1.0)
| | | age = [40-50]: died (1.0)
| serum_sodium = low
| | platelets = normal
| | | ejection_fraction = low
| | | | age = [61-70]
| | | | diabetes = yes: died (3.0)
| | | | diabetes = no
| | | | | high_blood_pressure = yes: died (1.0)
| | | | | high_blood_pressure = no: survived (2.0)
| | | | age = 70plus
| | | | diabetes = no: died (6.0)
| | | | diabetes = yes
| | | | | high_blood_pressure = yes: died (2.0)
| | | | | high_blood_pressure = no: survived (1.0)
| | | | age = [51-60]
| | | | | anaemia = no
| | | | | diabetes = yes: survived (2.0)
| | | | | diabetes = no: died (1.0)
| | | | | anaemia = yes: died (3.0)
| | | | age = [40-50]
| | | | sex = M: survived (2.0)
| | | | sex = F: died (2.0)
| | | ejection_fraction = mildly-low
| | | | CPK = high: died (2.0)
| | | | CPK = very-high: survived (1.0)
| | | | CPK = normal: died (2.0)
| | | ejection_fraction = borderline: survived (1.0)
| | | ejection_fraction = normal: survived (1.0)
| | platelets = low: died (5.0)
| | platelets = very-high: died (1.0)
| serum_sodium = high: died (1.0)
serum_creatinine = normal
| ejection_fraction = borderline
| | diabetes = no
| | | CPK = high
| | | | platelets = normal
| | | | age = [40-50]
| | | | | anaemia = no: died (1.0)
| | | | | anaemia = yes: survived (1.0)
| | | | age = 70plus: survived (1.0)
| | | | age = [61-70]: survived (1.0)
| | | | platelets = low: died (1.0)

```

```

| | | CPK = normal: survived (6.0)
| | | CPK = very-high: survived (1.0)
| | diabetes = yes
| | | anaemia = yes: died (3.0)
| | | anaemia = no
| | | | age = [61-70]
| | | | | CPK = high: survived (1.0)
| | | | | CPK = normal: died (1.0)
| | | | age = 70plus: survived (1.0)
| ejection_fraction = low
| | age = [40-50]
| | | platelets = normal
| | | | high_blood_pressure = no
| | | | | CPK = very-high
| | | | | | anaemia = no
| | | | | | | smoking = no: died (1.0)
| | | | | | | smoking = yes: survived (1.0)
| | | | | | | anaemia = yes: survived (2.0)
| | | | | | CPK = normal: survived (5.0)
| | | | | | CPK = high
| | | | | | | smoking = no
| | | | | | | | diabetes = no
| | | | | | | | | anaemia = yes: survived (1.0)
| | | | | | | | | anaemia = no: died (2.0)
| | | | | | | | | diabetes = yes
| | | | | | | | | anaemia = yes: died (1.0)
| | | | | | | | | anaemia = no: survived (4.0)
| | | | | | | | smoking = yes: survived (2.0)
| | | | | | | CPK = severely-high: survived (1.0)
| | | | | high_blood_pressure = yes
| | | | | | CPK = high
| | | | | | | diabetes = yes: survived (3.0)
| | | | | | | diabetes = no: died (1.0)
| | | | | | CPK = severely-high: died (1.0)
| | | | | | CPK = very-high: survived (1.0)
| | | | | | CPK = normal: died (2.0)
| | | | platelets = very-high: survived (1.0)
| | | | platelets = low
| | | | | anaemia = no: survived (1.0)
| | | | | anaemia = yes: died (2.0)
| | | | platelets = high: survived (2.0)
| | age = [51-60]
| | | platelets = normal
| | | | CPK = normal
| | | | | anaemia = yes
| | | | | | high_blood_pressure = no
| | | | | | | serum_sodium = normal: died (3.0)
| | | | | | | serum_sodium = low: survived (1.0)
| | | | | | high_blood_pressure = yes: survived (3.0)
| | | | | | anaemia = no: survived (6.0)
| | | | | CPK = very-high
| | | | | | serum_sodium = normal
| | | | | | | high_blood_pressure = no
| | | | | | | sex = M: died (1.0)
| | | | | | | sex = F: survived (1.0)
| | | | | | high_blood_pressure = yes: survived (1.0)
| | | | | | serum_sodium = low: died (1.0)
| | | | | CPK = high
| | | | | | anaemia = no: survived (2.0)

```

```

| | | | | anaemia = yes: died (1.0)
| | | | | CPK = severely-high: died (1.0)
| | | | | platelets = low: survived (4.0)
| | | | | platelets = high
| | | | | CPK = high: died (1.0)
| | | | | CPK = normal: survived (1.0)
| | | | | platelets = very-high: survived (1.0)
| | age = 70plus
| | | CPK = high
| | | | diabetes = no
| | | | | sex = M
| | | | | | anaemia = no
| | | | | | | serum_sodium = low
| | | | | | | | high_blood_pressure = no: died (1.0)
| | | | | | | | high_blood_pressure = yes: survived (1.0)
| | | | | | | | serum_sodium = normal: survived (2.0)
| | | | | | | | anaemia = yes: died (1.0)
| | | | | | | | sex = F: survived (2.0)
| | | | | | | | diabetes = yes
| | | | | | | | anaemia = yes: survived (2.0)
| | | | | | | | anaemia = no: died (2.0)
| | | | | | | | CPK = normal: died (3.0)
| | | | | | | | CPK = severely-high: died (1.0)
| | | | | | | | CPK = very-high: survived (1.0)
| | age = [61-70]
| | | CPK = high
| | | | smoking = yes
| | | | | high_blood_pressure = yes
| | | | | | anaemia = no: survived (2.0)
| | | | | | anaemia = yes: died (1.0)
| | | | | | high_blood_pressure = no: died (2.0)
| | | | | | smoking = no
| | | | | | anaemia = no: survived (6.0)
| | | | | | anaemia = yes
| | | | | | | diabetes = no
| | | | | | | | high_blood_pressure = yes
| | | | | | | | sex = F: survived (1.0)
| | | | | | | | sex = M: died (1.0)
| | | | | | | | high_blood_pressure = no: survived (1.0)
| | | | | | | | diabetes = yes: survived (1.0)
| | | | | | CPK = normal
| | | | | | high_blood_pressure = no
| | | | | | sex = M: survived (2.0)
| | | | | | sex = F: died (1.0)
| | | | | | high_blood_pressure = yes: survived (6.0)
| | | | | | CPK = very-high: survived (2.0)
| | ejection_fraction = normal
| | | age = [40-50]: survived (6.0)
| | | age = [61-70]
| | | | high_blood_pressure = yes: survived (7.0)
| | | | high_blood_pressure = no
| | | | | diabetes = no
| | | | | | anaemia = yes: died (1.0)
| | | | | | anaemia = no: survived (1.0)
| | | | | | diabetes = yes: survived (2.0)
| | | age = [51-60]
| | | | CPK = high
| | | | | diabetes = no: survived (1.0)
| | | | | diabetes = yes: died (2.0)

```



```

| | | CPK = normal: survived (6.0)
| | | CPK = very-high: survived (1.0)
| | age = 70plus: died (1.0)
| ejection_fraction = mildly-low
| | high_blood_pressure = no: survived (29.0)
| | high_blood_pressure = yes
| | | age = [40-50]: survived (4.0)
| | | age = [61-70]
| | | | anaemia = yes: survived (3.0)
| | | | anaemia = no: died (2.0)
| | | age = [51-60]: survived (3.0)
| | | age = 70plus
| | | | CPK = high: survived (1.0)
| | | | CPK = normal: died (1.0)
| | | | CPK = very-high: survived (1.0)
serum_creatinine = low: survived (1.0)

Number of Leaves : 128
Size of the tree : 215

```

Appendix D: MyDT+ Decision Tree Diagram

```

=== Classifier model (full training set) ===
MyDT+ tree
-----

serum_creatinine = high
| serum_sodium = normal: survived (39.0/17.0)
| serum_sodium = low: died (38.0/10.0)
| serum_sodium = high: died (1.0)
serum_creatinine = normal
| ejection_fraction = borderline: survived (18.0/6.0)
| ejection_fraction = low: survived (104.0/31.0)
| ejection_fraction = normal: survived (28.0/4.0)
| ejection_fraction = mildly-low: survived (44.0/3.0)
serum_creatinine = low: survived (1.0)

Number of Leaves : 8
Size of the tree : 11

```